21. Quantum LDPC instances with independently certified code parameters

Pure Thought research portfolio

Revised 17 September 2026

21. Quantum LDPC instances with independently certified code parameters

Revised specification — 2026-09-11. Detailed critique · Original description · Evidence standard

Plain-language guide

The problem in everyday terms

Quantum information is fragile, but simply copying an unknown quantum state is not allowed. An error-correcting code instead stores information jointly across many physical qubits so that selected measurements reveal errors without directly reading the encoded information. This problem builds sparse-check codes and verifies how much information they store, how many errors their structure can withstand, and how a chosen recovery algorithm performs.

Key terms

- **Qubit** — A quantum information unit whose state can involve superpositions of two basis states.
- **Physical and logical qubits** — The individual qubits used by the device, and the protected information units encoded across them.
- **Quantum error-correcting code** — A structured encoding that permits detection and correction of specified errors.
- **LDPC** — Low-density parity-check: a code family with checks involving bounded numbers of qubits and each qubit participating in bounded numbers of checks.
- **CSS code** — A code built from compatible binary checks for two types of quantum error, named after Calderbank, Shor and Steane.
- **Stabilizer** — An operator that leaves every valid encoded state unchanged; measuring checks can reveal violations.
- **Syndrome** — The collection of check outcomes indicating what kind of error may have occurred.
- **Code distance d** — The smallest number of qubits on which an undetectable nontrivial logical operation can act.
- **Decoder** — An algorithm that uses a syndrome to choose a recovery operation.
- **Hypergraph product** — A construction combining two classical parity-check matrices into a quantum code.

Why this matters

Useful quantum computation requires reliable logical information despite noisy components. Sparse checks offer a route to distributing error correction efficiently, but code parameters and decoder behavior must be demonstrated rather than assumed. Exact small examples are valuable for detecting mistakes that large simulations can conceal.

What progress would mean

Success would provide verified code parameters and decoder results under a stated error model. It would help compare constructions fairly, while leaving hardware connectivity, noisy measurements and the full cost of fault-tolerant computation as separate requirements.

Core question

For a specified CSS construction, can code dimension and distance bounds be verified independently, and can decoder performance be measured under one explicit noise model?

Scope and assumptions

Start with hypergraph-product codes from supplied binary parity-check matrices H_1 and H_2. Include the product of two full-rank 3×7 Hamming parity-check matrices as a small benchmark. Separate finite instances, asymptotic families, and noisy syndrome extraction.

Use independent X errors with perfect syndrome measurements for the first decoder experiment. A family is LDPC only when both row and column weights stay bounded as n grows.

Mathematical target

For H_i of size m_i×n_i define

$$H_X = [H_1 \otimes I_{n_2} \mid I_{m_1} \otimes H_2^{\mathsf{T}}], \quad H_Z = [I_{n_1} \otimes H_2 \mid H_1^{\mathsf{T}} \otimes I_{m_2}].$$

Check $H_X H_Z^{\mathsf{T}} = 0$ over F_2, n=$\mathrm{n}_1\mathrm{n}_2$+$\mathrm{m}_1\mathrm{m}_2$, $k = n - \operatorname{rank} H_X - \operatorname{rank} H_Z$. Define $d_X = \min\{|x| : x \in \ker H_Z \setminus \operatorname{row} H_X\}$ and d_Z analogously, $d = \min(d_X, d_Z)$. A found logical operator upper-bounds distance; ruling out all smaller logical operators lower-bounds it.

Required outputs and proof obligations

Export sparse matrices, GF(2) row reductions, logical bases, explicit upper witnesses and lower-bound proofs. SAT certificates need a verified encoding of nontrivial homology and weight. Decoder experiments report physical noise, recovery rule, logical-failure test, sample counts, confidence intervals, and runtime.

Validation and rejection controls

For the two 3×7 inputs verify n=58 and k=16; certify the claimed distance independently rather than assuming it. Inject all small-weight errors on tiny codes. A matching syndrome is not success unless error plus recovery is a stabilizer. Compare optimized checks with an independent GF(2) implementation.

Milestones and research extension

Baseline: exact small product code and certified distance bracket. Strong: several instances plus honest finite-size decoder curves. Research extension: a new family tradeoff or proved decoder guarantee. Reproducing asymptotically good codes is not itself a new existence result.

Milestones are acceptance gates, not calendar promises. Before a run, freeze the concrete instance/domain, tolerance, compute budget, and permitted dependencies. A bounded search that fails to find a result must preserve an unresolved outcome.

Starting instruction

Work on the baseline above. First make every input and convention explicit, then validate the reference case. Build the checker alongside the calculation. Label numerical evidence, finite-model certificates, and continuum theorems separately; return the strongest justified result and identify the exact unmet proof obligation.

Primary references and their role

- Panteleev and Kalachev, Asymptotically Good Quantum and Locally Testable Classical LDPC Codes — Existing asymptotically good quantum LDPC constructions; novelty baseline.

References support definitions or limitations; the scope choices and proposed milestones are editorial recommendations, not claims that these results have already been obtained.

Displayed formulation and claim boundary

Canonical specification.

$$H_X = [H_1 \otimes I_{n_2} \mid I_{m_1} \otimes H_2^{\mathsf{T}}], \qquad H_Z = [I_{n_1} \otimes H_2 \mid H_1^{\mathsf{T}} \otimes I_{m_2}],$$

$$H_X H_Z^{\mathsf{T}} = 0, \quad n = n_1 n_2 + m_1 m_2, \quad k = n - \operatorname{rank}_{\mathbb{F}_2} H_X - \operatorname{rank}_{\mathbb{F}_2} H_Z,$$

$$d_X = \min_{x \in \ker H_Z \setminus \operatorname{row} H_X} |x|, \qquad d_Z = \min_{z \in \ker H_X \setminus \operatorname{row} H_Z} |z|, \qquad d = \min(d_X, d_Z).$$

Two full-rank 3×7 Hamming checks give $n = 58$, $k = 16$; the included independent validator also checks $d = 3$. A logical witness gives an upper distance bound; exclusion of all smaller logicals gives the lower bound. Finite code parameters do not establish an asymptotic LDPC family or decoding threshold.

Detailed review and rationale

Revised problem · Preserved original · Pinned upstream source

Line references below refer to the preserved original at commit `f854d4e12b55`, not the rewritten file. "Blocking" means the original issue can invalidate the scientific target or claimed certificate; "major" means it materially changes scope, inference, or acceptance criteria. These are critiques of specifications, not results of running the embedded research-code sketches.

Detailed findings

1. **Blocking — wrong concrete size (974–986).** Two 3×7 parity-check inputs give $7^2+3^2=58$ qubits, not 90. The construction and ranks should determine the parameters.
2. **Major — distance shortcut (92–96, 60).** Dimension follows from linear algebra; minimum nontrivial logical weight generally requires harder optimization. The displayed dual-code distance shortcut is not an adequate universal CSS certificate.
3. **Major — finite versus asymptotic (38–42, 995–1005).** A few codes with good ratios cannot prove constant rate, linear distance, or a threshold. Ordinary hypergraph products do not automatically yield linear distance.
4. **Major — noise and overhead.** Pauli labels are discrete even though general quantum noise is richer. Decoder threshold and fault-tolerant overhead require a noise and measurement model. The rewrite removes unsupported hardware-overhead comparisons and recognizes existing asymptotically good constructions.

What the revision preserves and changes

The central motivation is retained, but the executable question is now: **For a specified CSS construction, can code dimension and distance bounds be verified independently, and can decoder performance be measured under one explicit noise model?** The revised scope chooses a concrete starting model; those choices are proposed research-design decisions, not facts inferred from the original. The baseline can succeed without resolving the full open research program.

The replacement also removes the original implementation sketches from the active specification. They remain in the archive for comparison, but have not been made executable or certified. New implementations should derive their equations from the revised definitions rather than copy unchecked prototypes.

Acceptance criteria to use instead

Baseline: exact small product code and certified distance bracket. Strong: several instances plus honest finite-size decoder curves. Research extension: a new family tradeoff or proved decoder guarantee. Reproducing asymptotically good codes is not itself a new existence result.

Sources supporting the corrections

- Panteleev and Kalachev, Asymptotically Good Quantum and Locally Testable Classical LDPC Codes — Existing asymptotically good quantum LDPC constructions; novelty baseline.

Evidence and execution standard

This standard applies to the revised specifications. It is a proposed contract for future research runs; this editorial revision does not supply the scientific certificates themselves.

What "pure thought" means here

Use explicitly stated mathematical models and deductions without fitting conclusions to experimental outcomes. Synthetic examples, exact model inputs, analytic references, numerical search, and established software are allowed. Physical constants, electronic-structure approximations, constitutive laws, and empirical parameter sets must be identified when used. A model-derived result is conditional on that model.

The original ban on legacy software conflicts with its proposed reliance on NumPy, SciPy, CVXPY, nauty, and other libraries. Reimplementing every optimizer and algebra system is not necessary for scientific independence. Prefer a small independently checkable certificate and a documented trusted base. A fresh implementation can be a project objective, but must not be confused with a mathematical proof or novelty claim.

Four evidence levels

Label	What it establishes	What it does not establish
Numerical evidence	A reproducible computation at stated precision, sample size, and discretization	Exact truth, continuum coverage, or a universal theorem
Finite-model certificate	An exact/validated statement about the specified finite model or finite relaxation	An omitted tail, thermodynamic limit, or physical realization
Conditional mathematical theorem	A proof over the stated domain with every hypothesis and error controlled	Truth outside those assumptions or a demonstrated physical system
Formal proof	A checked theorem in a named proof assistant, with imported axioms/dependencies listed	Correct physical modeling merely by virtue of formalization

Formalization is an optional extra deliverable unless the specific project makes it mandatory. High precision, JSON output, solver status and small residuals do not by themselves confer any of these proof levels.

Claim directions

For minimization over a larger feasible set, the optimum is a lower bound on the original minimum. A physical feasible point supplies an upper bound. For maximization the directions reverse. Track feasibility of the primal/dual witnesses separately from their objective values.

Infeasibility of an inner ansatz excludes only that ansatz. Infeasibility of an outer relaxation can exclude the original problem if the map into the relaxation is proved and all approximations preserve necessity. Passing necessary tests means "not excluded"; existence needs a constructive sufficient argument.

A smallest observed example is not a minimality theorem. A complete catalog must define the universe, equivalence relation, boundaries, and proof that pruning misses nothing. A finite-size numerical trend does not prove an asymptotic limit.

Minimum run manifest

Before calculation, record the problem ID and revision hash, exact inputs and conventions, quantified domain, equivalence relation, target observable, accepted evidence level, tolerances, and resource limits. Record dependencies and precision/rounding mode. Where randomization is used, preserve seeds, trial counts and raw sufficient statistics. Resource limits are planning decisions, not evidence that a mathematical solution cannot exist.

An output bundle should include:

- The statement being claimed, including all assumptions and quantifiers.
- Construction or proof object, plus an independently runnable verifier and its trust assumptions.
- Numerical/error ledger separating roundoff, discretization, truncation, optimization, sampling and modeling error where relevant.
- Positive reference cases and negative controls that the verifier must reject.
- Coverage: included domain, omitted domain, unresolved regions and termination reason.

These are requirements for future implementations, not a promise of polynomial-size certificates for every problem.

Checking specific certificate types

LP/SDP/SOS: verify equality constraints, objective direction, and cone feasibility using exact arithmetic or outward-rounded intervals. Rationalizing a numerical solution can break PSD constraints. Correct residuals with justified norm bounds; do not silently project and reuse the old objective. Distinguish a proof of a polynomial approximation from a proof for the original function.

Interval numerics: state function enclosures, domains and outward rounding. A small root box needs existence and, if claimed, uniqueness. Cover the complement for a global enumeration. Include singularities and tails rather than discarding them numerically.

SAT/SMT: a solver's UNSAT label is not a certificate. Preserve a proof supported by a checker and the proof that the encoding, symmetry reduction and blocked solutions match the mathematical universe. DRAT applies to an appropriate Boolean CNF proof, not automatically to every real-arithmetic or SDP claim.

Statistics: specify the estimand, dependence assumptions, confidence procedure and bias. Zero observed failures does not prove zero failure probability. MCMC error bars based on estimated autocorrelation are not generally rigorous mixing guarantees.

Milestones and novelty

Baseline means a working, independently validated result on a concrete reference problem. Strong means a broader or sharper result with its claimed evidence level. Research extension means a possible novel result whose feasibility and significance need review. A negative theorem can be valuable; an unsuccessful search can also be useful if its scope is documented, but the two are not interchangeable.

Do not promise "publication-quality" in a fixed number of months. Before claiming novelty, conduct a focused current literature comparison against the exact theorem/model/metric. The sources in

this revision check key definitions and limitations; they are not an exhaustive novelty audit across all 30 fields.